CCF 软件大会 2026 · Rust 软件原型开发竞赛

参赛作品技术文档

AgentFence Lite

基于 Rust 的 AI Agent 文件操作权限网关

让 Agent 的每一次文件读写，都可授权、可审计、可复现。

评审版本　ccf2026-final

对应提交　f1193a6

项目仓库　github.com/shishuishuia/AgentFence

文档日期　2026 年 10 月 30 日

团队与成员信息见报名表。

摘要

AgentFence Lite 是一个基于 Rust 的 AI Agent 文件操作权限网关。它不生产 Agent，而是为外部 AI Agent 的文件工具调用提供统一入口：每次读写在服务端依次通过身份、阶段、精确路径与写租约四项判定，执行前后分别写入 intent 与 result 审计；工作区以不可变快照或隔离副本的形式提供，修改必须由管理员逐文件确认后才写回原项目。当前版本通过 MCP stdio 接入 DeepSeek Harness，Rust 与前端测试合计 170 项、Python 单测 162 项，主分支 CI 全绿，评审版本为 tag ccf2026-final。

关键词：AI Agent；最小权限；MCP；写租约；审计；Rust

1 项目简介

1.1 一句话定位

把“Agent 能做什么”从提示词里的承诺，变成服务端可执行、可验证的约束。

在大模型开始直接修改代码的今天，团队真正缺的不是更强的 Agent，而是让 Agent 安全落地的中间层：既能表达“这个任务、这段时间、只能改这个文件”，又能在事后拿出逐条证据。AgentFence Lite 就是这一层。

1.2 一页看懂

维度	说明
面向对象	需要让 AI Agent 读写本地代码，但不愿直接授予文件系统权限的团队与个人
交付形态	Rust 工具网关 + MCP stdio 适配器 + Vue 控制台 + 本地只读代理
核心约束	阶段权限、精确路径白名单、限时限次写租约、intent/result 双审计
可验证性	快照 SHA-256、租约状态快照、request_id 请求关联、按实例隔离的事件游标
运行方式	本机回环；Admin Token 与管理接口隔离；外部 Agent 只获得任务级 Token

1.3 它不是 Agent

项目刻意不实现模型推理与任务规划：Agent 可以是 DeepSeek Harness、其他 MCP 客户端，甚至自研运行时。AgentFence 只负责三件事——把文件操作收敛到唯一入口、在入口处做权限判定、

把判定与执行结果写成可核对的证据。内置执行器仅用于隔离修改的过渡路径和断网演示，不作为产品的 Agent 能力宣传。

2 研发背景与问题定义

2.1 从“读”到“写”的风险跃迁

只读场景下，Agent 的越权最多带来信息泄露；一旦获得写权限，一次错误的路径、一次过期未撤销的授权、一次并发覆盖，都可能造成不可逆的损失。传统做法把风险寄托在三处：提示词里的“不要改别的文件”、前端隐藏的写按钮、以及人工盯着看。这三者都无法在事后证明“它确实没做”。

2.2 现有约束为何不够

- 前端隐藏：只是界面行为，直接调用 HTTP 或 MCP 即可绕过。
- 提示词约束：是概率性的，模型不遵守时没有强制手段，也没有拒绝记录。
- 操作系统沙箱：隔离强，但部署重，且难以表达任务语义——“这个任务在 60 秒内只能写这个文件一次”。
- 通用策略引擎：擅长表达“谁能否访问什么”，但与文件执行、租约生命周期和审计证据是分离的。

因此需要一层位于 Agent 与文件系统之间的“任务感知”中间件：权限随任务创建、随阶段变化、随租约到期而失效，并且每次判定都留下证据。

2.3 设计目标

1. 服务端判定：所有授权在 Rust Broker 内完成，前端隐藏不构成安全。
2. 最小权限可表达：精确到相对路径、有效秒数与使用次数。
3. 全链路可审计：意图与结果成对记录，可按请求关联、按实例隔离。
4. 不侵入 Agent：以 MCP 标准接入，外部运行时无需改代码。

2.4 明确的非目标

不实现操作系统级沙箱、通用 MCP 透明代理、多 Agent 调度、Shell 执行、网络代理或模型推理优化。它保护的是“经过 Broker 的调用”，而不是整个 Agent 进程。这一边界在文档、界面与答辩中都被明确标注，避免把工具网关宣传成系统级隔离。

3 核心功能

3.1 四阶段任务模型

任务从创建起绑定一个工作区快照和一种权限配置。阶段只能按顺序推进，Agent 无法自行跳转；只读任务的合法路径是 inspect → verify → done，永远不会进入 edit。

阶段	允许的工具	说明
inspect	read_file	读取已批准的快照文件，不可写
edit	read_file / write_file（需租约）	仅 controlled_edit 任务；写操作必须持有绑定该路径的有效租约
verify	read_file	读回修改结果，核对内容与哈希
done	无	任务关闭，任何工具调用都被拒绝

3.2 快照、只读任务与隔离副本

导入项目时，管理员先预览允许与排除的文件，确认后由 Broker 生成不可变快照；预览 300 秒有效且只能确认一次，源文件变化会被 SNAPSHOT_CHANGED 拒绝。默认上限为 1000 个文件、单文件 64 KiB、总计 20 MiB、20000 个遍历项与 64 层目录，超限明确报错而不是静默截断。

只读任务只读快照；受控修改任务基于快照创建独立副本，模型只能改副本中的已授权现有文件。任务完成后，管理员逐文件查看原内容与修改后内容，确认后才写回原项目；写入前会重新校验副本哈希与原文件是否仍与导入时一致，冲突即拒绝。

3.3 外部 Agent 接入

外部 Harness 通过 MCP stdio 调用三个只读工具：列出文件、读取文件、查询任务状态；在隔离副本任务中额外开放写工具。Agent 进程拿不到任务 Token——它只看到命名管道名，由本机中继在转发时注入凭证。守护插件把可用工具收敛到白名单，模型自带的文件与 Shell 工具被拒绝。

3.4 审计与可关联

每次调用在审计中至少留下两条记录：执行前的 intent 与执行后的 result，共享同一个 request_id。前者说明“打算做什么”，后者说明“是否允许、是否执行”。管理员操作（阶段推进、租约签发/撤销、工作区导入）同样带 request_id，因此状态变化也能对回审计。

3.5 控制台

Vue 控制台提供工作区导入、任务列表与详情、差异确认、审计查询和模型凭证入口。审查进行时按秒刷新状态；其余页面每 60 秒轮询，避免字段闪烁。凭证面板只写不读：页面只显示“已配置/未配置”，密钥保存到服务端用户目录，不回显、不写入浏览器存储。

4 技术实现

4.1 分层与职责

模块	职责	关键设计
crates/core	纯策略：阶段、路径、租约、错误码	无 HTTP、无文件 I/O，可单独基准
crates/server	HTTP、任务状态机、文件执行、审计	Axum；任务级锁；原子替换
crates/mcp-server	MCP stdio 适配	官方 rmcp 3.8.0；只转发，不直接读写文件
front/	Vue 3 控制台	轮询与实例恢复；差异逐项确认
web_proxy/	本地管理代理	仅回环；同源/CSRF；不向浏览器吐 Token

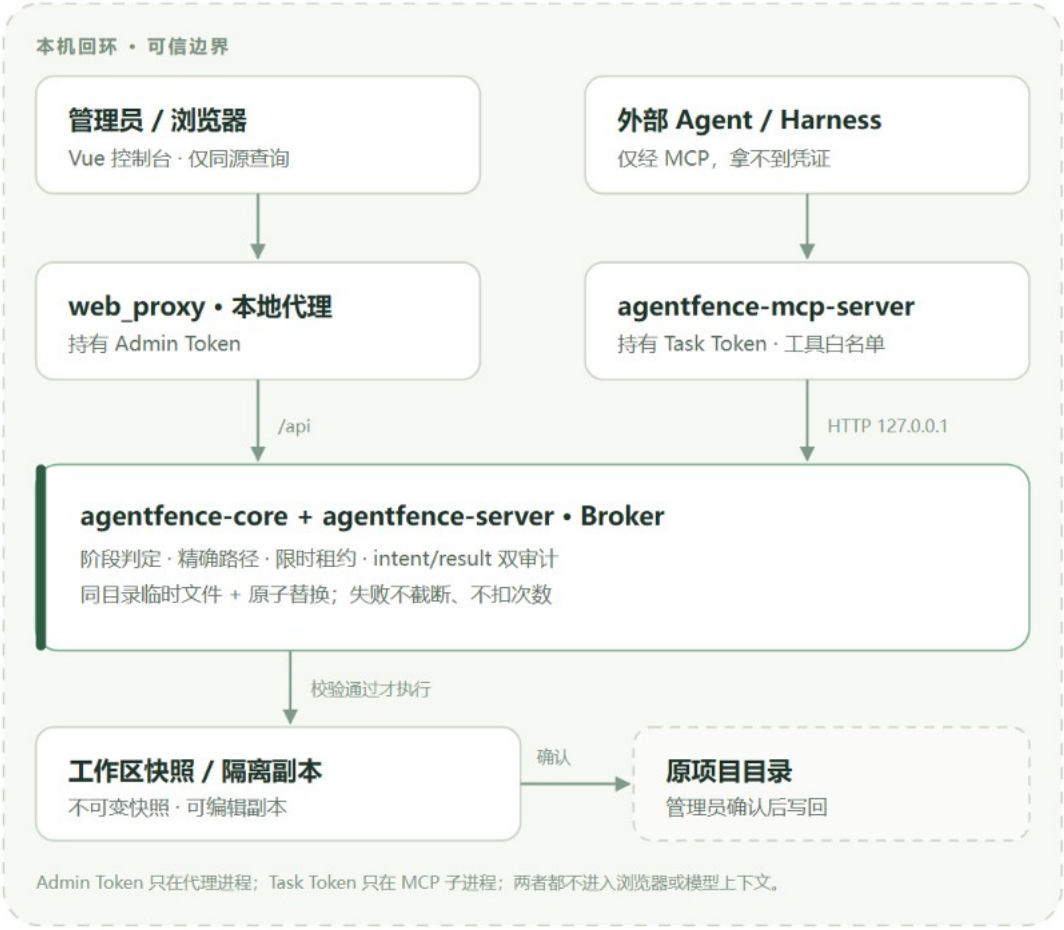


图 1　分层架构与信任边界：外部 Agent 只经 MCP 访问 Broker，凭证不进入浏览器或模型上下文。

4.2 一次工具调用的判定流水线

1. 认证：Admin Token 与任务 Token 分离；只存 SHA-256 摘要，常量时间比较。

2. 阶段：done 一律拒绝；write_file 仅在 edit 阶段允许。
3. 路径：先做可移植语法校验，再做白名单精确匹配，最后逐段检查符号链接、重解析点与普通文件，并比较规范路径前缀。
4. 租约：必须显式提供 grant_id；按“存在→绑定→撤销→过期→次数”的固定顺序判定。
5. 执行：同目录临时文件写入、复制原权限、fsync，再原子替换目标文件。
6. 审计：intent 先落盘，result 后落盘；审计不可用时关闭后续执行，避免“改了但没证据”。

判定与执行在同一任务锁内完成，文件 I/O 与审计放在阻塞线程池，避免占用异步运行时。

4.3 写租约

租约绑定任务、工具与精确路径，有效期 1–300 秒、最多使用 1–10 次。过期使用单调时钟判定，不受系统时间调整影响；展示用的 UTC 截止时间只用于界面。租约状态按“阶段失效 → 撤销 → 过期 → 次数耗尽 → 有效”的优先级计算，成功写入后扣减一次，失败或拒绝不扣减。

4.4 原子写入与审计一致性

写入使用同目录临时文件，保留原权限并 fsync 后原子替换。替换失败时临时文件被清理，目标文件保持原样，租约也不扣减。审计采用按行追加的 JSONL；若 result 写入失败，服务进入失败关闭状态，后续工具调用被拒绝，直到人工检查并重启。

4.5 外部 Agent 的租约注入

模型调用写工具时只提供 path 与 content，不提供 grant_id。管道中继解析 JSON-RPC 帧，在转发前按路径向 Broker 申请一次性租约（TTL 120 秒、1 次），把 grant_id 注入后原样转发给 MCP 子进程；模型伪造的 grant_id 会被覆盖。租约申请失败时原样转发，由 Broker 返回 LEASE_REQUIRED。模型全程接触不到任务 Token 与管理员凭证。

4.6 并发与容错

每个任务持有独立互斥锁，授权、文件操作与租约扣减在同一临界区内完成。单次写入在 10 个并发请求下只会有一个成功；撤销与过期不会取消已进入替换阶段的写操作，但会阻止后续写入。请求超时或断连不能推断“未写入”，调用方必须核对文件、租约与审计后再决定是否重试。

5 Rust 的核心作用

5.1 为什么选择 Rust

这个项目对“确定性”的要求高于对开发速度的要求：权限判定不能被 GC 暂停打乱，租约过期必须与单调时钟严格对应，文件替换必须可控且可清理，审计失败必须立即关闭执行。Rust 的所有权与类型系统让这些约束在编译期成立，而不是靠代码评审去保证。

- 无 GC：延迟可预测，权限判定是纯函数，基准可稳定复现。
- 所有权与借用：任务、租约、审计存储的并发访问由类型系统约束，避免数据竞争。
- Result 与显式错误：每个拒绝都有稳定错误码，不以异常或字符串表达安全结果。
- 零成本抽象：策略层保持纯函数，HTTP 层与执行层可独立测量。

5.2 以类型表达安全约束

工作区声明 #![forbid(unsafe_code)]，缺失文档会产生警告；阶段、权限配置、租约状态、错误码都是枚举而非字符串，未知取值无法反序列化。路径校验、租约判定、审计写入分别由独立模块承担，调用方必须显式组合，不能“忘记检查”。

5.3 实测性能

场景	指标	观测值
纯策略判定（allow / deny）	P50	0.093 µs / 0.024 µs
HTTP 读取（enforced，4 KiB）	P50	0.866 ms
HTTP 写入（enforced，4 KiB）	P50	1.346 ms
MCP stdio 读取 vs 直接 HTTP	P50	5.63 ms vs 4.31 ms（+1.32 ms）

以上数据分别来自纯策略微基准、Ubuntu CI 配对采样与 MCP 适配开销实验；均为单机观测，包含 HTTP、文件与同步审计，不构成生产性能承诺。

5.4 可复现的工程

依赖由 Cargo.lock 与 package-lock.json 固定，工具链由 rust-toolchain.toml 固定为 1.98.1；CI 覆盖 Linux 与 Windows 的格式、Clippy、测试、文档与 release 构建，并运行黑盒、性能与权限窗口实验。仓库以 MIT 发布，第三方依赖与许可证索引一并归档。

6 效果评估

6.1 测试矩阵

层次	数量	覆盖内容
Rust	102 项	策略与路径、HTTP 契约、快照与只读、隔离副本、MCP stdio、并发与审计故障
前端	68 项	轮询与实例恢复、审计分类、工作区导入、任务与凭证、错误与竞争
Python	162 项	HTTP 客户端、只读代理、演示流程、基准工具（集成 62 / 代理 38 / 演示 45 / 基准 17）
黑盒	34 场景	A01–A10 与 D01–D24：鉴权、阶段、路径、租约、分页、重启与并发

主分支 CI 的 Dashboard、Rust 与 V2 B 三条流水线均为 success；评审版本 tag 指向提交 f1193a6。

6.2 权限窗口实验

在同一台 Ubuntu 24.04 / WSL2 机器上，以 6 秒静态授权为基线，对比动态授权实际生效的区间长度。动态模式实际执行阶段与租约约束：

场景	静态时长	动态观测（3 轮）	缩减率
租约 TTL 到期	6.000 s	3.97–4.04 s	约 33.3%
离开 edit 阶段	6.000 s	1.79–1.81 s	约 70.0%
撤销/再授权后离开 edit	6.000 s	3.28–3.32 s	约 45.0%
写入次数耗尽	6.000 s	1.88–1.95 s	约 68.1%

6.3 HTTP 配对实验

场景	enforced P50	baseline P50	样本
读取	0.866 ms	0.858 ms	各 300
写入	1.346 ms	1.336 ms	各 300
策略拒绝	0.616 ms	0.603 ms	各 300

两种模式复用同一 HTTP、文件与审计实现，仅授权步骤不同；三轮差值有正有负，不能据此声称策略开销或性能优势。

6.4 真实模型运行

2026 年 10 月 29 日，以 DeepSeek Harness（deepseek-flash）对一份包含 14 个文件的 Python 练习项目执行只读审查：模型经 MCP 读取 13 个文件，生成 9426 字符报告，审计中 13 条 read_file 结果全部 allowed 且 executed，request_id 与报告引用一一对应。随后一次隔离修改任务只改动一个文件，写入租约由启动器签发，模型未接触凭证；任务在 verify → done 后交由管理员逐文件确认。两次运行均由真实 DeepSeek API 驱动，而不是离线复放。

6.5 资源占用

在 34 个黑盒场景的完整负载下，Rust 服务进程的 RSS 峰值为 5372 KiB（约 5.2 MiB），说明网关本身的资源开销很小，适合与本机 Agent 同时运行。

7 相关工作与差异

方向	代表做法	与本项目的差异
MCP 生态	官方 SDK、各类 MCP Server	多数只负责暴露工具；本项目在其前面加任务级租约、路径限制与双审计
策略引擎	OPA / Cedar	通用策略语言，与文件执行和租约生命周期解耦；本项目把策略绑定到一次具体写入
操作系统沙箱	Firecracker / AppContainer	隔离更强，但部署重且无法表达“任务内 60 秒单次写”这类能力约束
终端 Agent 权限	交互式批准、白名单目录	以人工确认或目录粒度为主；本项目提供可编程、可审计的时间与次数边界

本项目的定位不是替代沙箱，而是补上“任务语义”这一层：沙箱回答“进程能碰到什么”，AgentFence 回答“这个任务此刻被允许做什么，做过什么”。两者可以叠加使用。

8 安全边界与已知限制

- 只保护经过 Broker 与 MCP 的调用；Agent 若自带未禁用的文件或 Shell 工具，仍可绕过网关直接访问宿主机。
- 路径检查不是无竞态的安全打开：不抵御同权限进程的并发替换、硬链接攻击或 TOCTOU。
- 外部 Harness 的信任边界是当前操作系统用户；命名管道只限制其他用户访问。
- 服务重启后任务、Token 与租约全部失效；JSONL 审计保留，但不在线恢复。
- 审计文件尾行不完整会阻止启动；写入超时后不能推断“未写入”，必须人工核对。

- 性能与窗口数据来自特定机器与提交，不代表其他环境或生产负载。

这些限制写在威胁模型、接口文档和界面提示中，并在答辩中主动说明，而不是等评委追问。

9 总结与后续

AgentFence Lite 用 102 项 Rust 测试、68 项前端测试、162 项 Python 测试和 34 个黑盒场景，验证了一条可落地的路径：把 Agent 的文件操作收敛到服务端网关，用阶段、路径、租约与审计四道约束表达最小权限，并用真实模型运行证明它可与现成 Harness 协同工作。

后续工作包括：任务与审计的持久化恢复、Linux 原生运行与更细的进程隔离、面向团队的策略描述格式、更多 Agent 运行时适配，以及在真实开发流程中积累使用数据。

附录 复现方式

仓库：github.com/shishuishuia/AgentFence　　评审版本：tag ccf2026-final（commit f1193a6）

最小验证：cargo test -p agentfence-server --test tcp --locked，成功输出 BACKEND_TCP_MINIMAL_PASS。

完整演示：构建 target/v2 与 front/dist 后运行 python -m demo.run_v2_dashboard，访问 127.0.0.1:4174/workspaces。

外部 Agent：按 docs/external-harness-integration.md 准备固定版本 Harness，运行 --harness-check 验证 ACP 握手。